# AI-Powered Smart Orchid Care System for Vanda Cultivation

Multi-Modal Machine Learning for Orchid Cultivation

**Project ID: R26-SE-018**

## DEPLOYMENT REPORT

B.Sc. (Hons) Degree in Information Technology

Specialising in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

**October 2026**

# TABLE OF CONTENTS


1 Introduction
2 Key Capabilities
3 Architecture Overview
4 Data Flow
5 System Components
  5.1 Component 1 – Hybrid Pollination and Compatibility Analysis
  5.2 Component 2 – Disease Detection and Severity Grading
  5.3 Component 3 – Smart Watering and Automated Fertilisation
  5.4 Component 4 – Growth Stage Classification and Bloom Timing
  5.5 Mobile Application and Field Hardware
6 Prerequisites
7 Deployment Options
8 Quick Start
9 Step-by-Step Instructions
10 Service and Port Map
11 Integration Guide
12 Configuration
13 Verification and Testing
14 Machine-Learning Model Reference
15 Troubleshooting
16 Cloud Deployment
17 Maintenance and Operations
18 Scaling
19 Support and Resources
20 Conclusion

# 1 INTRODUCTION

The AI-Powered Smart Orchid Care System is a research platform for Vanda cultivation that brings four independent machine-learning components together behind one mobile application and one application programming interface. It addresses the four judgements a grower makes repeatedly and at present makes by eye: whether a plant is diseased, when it will flower, how much water and fertiliser a bench requires, and whether a plant is fit to breed from and what it may be crossed with.

The system is deployed as a single FastAPI service on a Contabo virtual private server, reached by a React Native application over HTTPS. The four components share that process, one Firebase Realtime Database and one model directory rather than running as separate services. This is a deliberate decision for a project of this size. It removes inter-service networking, allows one component to call another in process where that is cheaper than a round trip, and reduces a deployment to a repository pull and a service restart rather than an orchestration exercise.

What the deployment must protect is the boundary between components, not the boundary between processes. Each component owns its router prefix, its model artefacts and its subtree of the database, and every point at which one component reaches into another is enumerated in Section 11.

The research question the platform addresses is whether photographs, low-cost sensor telemetry and a registered breeding record supply enough evidence to support those four decisions, and whether a system can be honest about the decisions it cannot support. The second half of that question shapes several of the deployment behaviours described below, because a component that declines to answer must be deployed in such a way that its refusal is visible rather than silently absorbed.

# 2 KEY CAPABILITIES

The pollination component assesses breeding readiness from a whole-plant photograph and returns a three-class verdict in which every contributing trait is

reported beside its origin, so that a grower can see which values were measured and which were supplied. Ahead of that classifier sit two gates: a one-class novelty detector and an orchid flower filter, which together refuse an out-of-domain image rather than extrapolating on it. All 309 non-plant images in the validation set are refused. The same component resolves cross compatibility against the registered hybrid record, returning an evidence tier and the precedents behind it rather than a success percentage, for reasons set out in Section 14, and maintains a crossing logbook in which attempted crosses are recorded with both parents photographed, advanced through the stages a capsule passes, and closed at the gate at which they stopped.

The disease component classifies a leaf close-up and grades the severity of a disease it has identified, and that classifier is consumed in process by the pollination component rather than reimplemented. The growth component recognises six growth stages and regresses the number of days remaining before bloom from image and environmental features. The watering component converts sensor telemetry into irrigation and fertilisation plans, applying kriging across a sparse sensor field so that a bench without its own node is planned from its neighbours rather than from defaults.

Two further capabilities are properties of the deployment rather than of any one component. Every farm path is rewritten onto the calling tenant at a single chokepoint, which the automated test suite asserts rather than assumes, so one farm's data cannot be served to another. And field capture is offline tolerant: logbook entries are written to local storage first and mirrored to the database second, so that a record made in a shade house with no signal is not lost.

# 3 ARCHITECTURE OVERVIEW

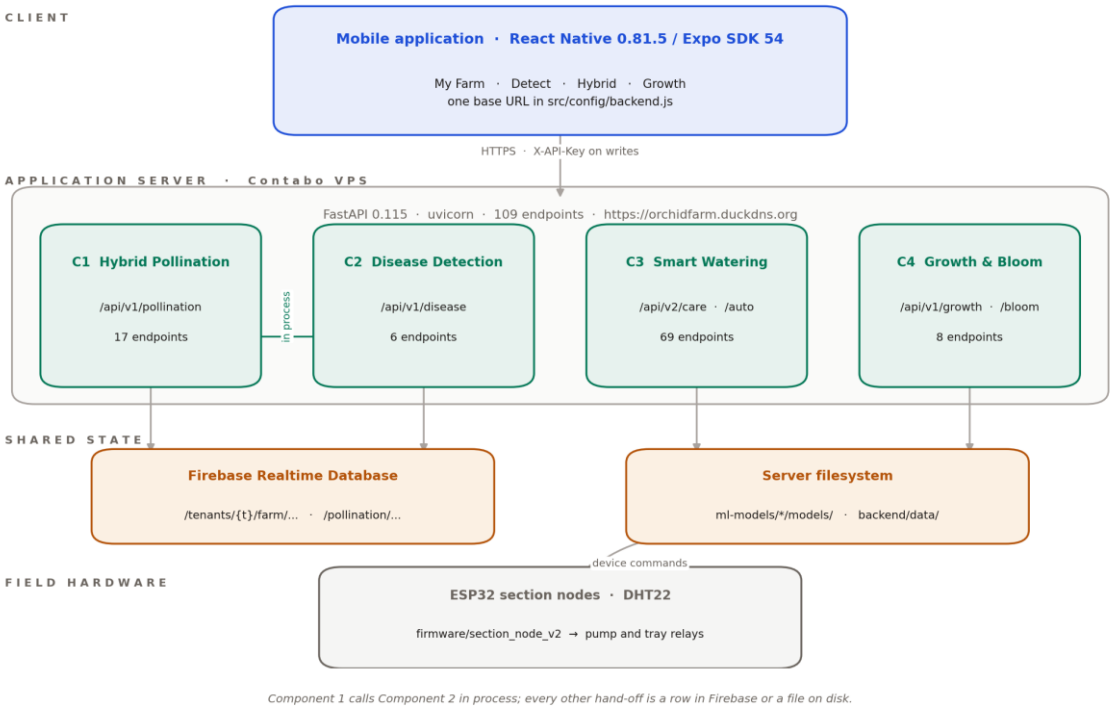



**Figure 1: Deployment architecture. One FastAPI process hosts all four components; state is shared through Firebase and the server filesystem.**

Three properties of this arrangement matter at deployment time. The application holds no state that is not also written to the database, so restarting the service loses nothing. Models are loaded once at process start and held in memory, so the first request after a restart is no slower than the hundredth. And the mobile application reads one base address from one file, so relocating the backend is a single-line change rather than a search through every screen that makes a request.

# 4 DATA FLOW

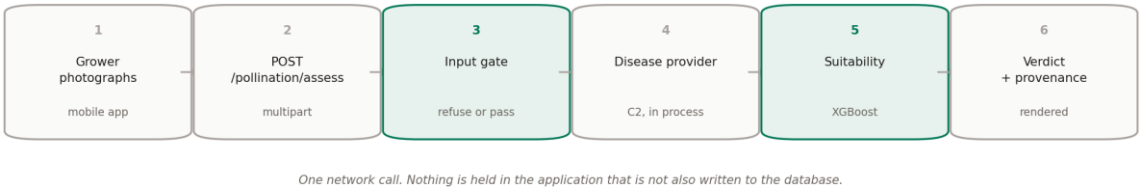



**Figure 2: One pollination assessment, from the photograph to the rendered verdict.**

The fourth step is the point at which the components stop being parallel. The pollination component requires a judgement on whether the plant before it is diseased, which is precisely the question the disease component exists to answer, so it calls that classifier through a provider interface rather than training a second disease model on its own smaller dataset.

The failure behaviour of that call is as much a part of the contract as its success behaviour. When the disease model is unavailable the provider reports zero confidence and the pollination component proceeds on image evidence alone; it does not treat an unidentifiable leaf as a healthy one. An earlier implementation checked only whether the model file existed, with the consequence that on a server lacking the deep-learning runtime the system fell back silently to a colour heuristic while continuing to report that the trained classifier was in use. The corrected check, which verifies that the runtime can actually be imported, is now the pattern used for every optional dependency in the backend.

# 5 SYSTEM COMPONENTS

All four components are routers within one FastAPI application defined in backend/app/main.py, and the live interface exposes 109 endpoints. Each is described below in the same terms: its purpose, where its code and artefacts reside, how it works, the interface it presents, and what it leaves on disk. Table 1 states the ownership of each router prefix.

| Component | Router prefix | Endpoints | Owner |
|---|---|---|---|
| Component 1, Hybrid Pollination | /api/v1/pollination | 17 | Wickramasinghe D. P. (IT22065230) |
| Component 2, Disease Detection | /api/v1/disease | 6 | Rathnamalala D. M. J. P. (IT22064936) |
| Component 3, Smart Watering | /api/v2/care, /devices, /auto, /accounts, and /api/v1/watering | 74 | Vithanage P. V. P. M. |
| Component 4, Growth and Bloom | /api/v1/growth, /api/v1/bloom | 8 | Component 4 owner |

**Table 1: Router ownership. Four further endpoints at the application root bring the total to 109.**

## 5.1 Component 1 – Hybrid Pollination and Compatibility Analysis

This component decides whether a plant is fit to breed from, whether a proposed pairing of two named parents has any precedent behind it, and records what actually happened when the cross was attempted. Its router is backend/app/api/routes/hybrid_pollination.py, supported by three services and by the model code under ml-models/hybrid_pollination/src/.

The assessment path runs in three stages. A frozen ResNet18 converts the photograph into a 512-dimensional embedding, over which a one-class nearest-neighbour detector refuses any image whose cosine distance to its tenth nearest neighbour among 1,190 reference photographs exceeds 0.25. An orchid flower filter then separates orchid blooms from other flowers, and only an image passing both gates reaches the classifier, where 66 image and trait features are sorted by XGBoost into Suitable, Moderate or Not Suitable.

Compatibility is resolved separately and from the registered record rather than from images. Each parent name is resolved by one of three routes, and the component

reports which was taken: the genus may be stated on the name, it may be absent but recorded in the knowledge base, or it may be neither, in which case the collection's genus is assumed and the result says so. A hybrid genus name is expanded into the natural genera it contains before any comparison, so that a cross back into a parent genus is recognised as such. The crossing logbook records attempts through an eight-stage pipeline with five exit states, reports counts beside their denominators, and suppresses rates until a gate has at least five completed attempts behind it.

The component is built on PyTorch 2.3.1 for the frozen embedding, XGBoost 2.1.1 for the classifier, and scikit-learn 1.5.0 for the gate and trait models. Its principal endpoints are given in Table 2.

| Method and path | Purpose |
|---|---|
| POST /api/v1/pollination/assess | Suitability from a photograph, with an optional leaf close-up and trait corrections |
| POST /api/v1/pollination/compatibility | Evidence tier for a directional pairing, with name resolution reported |
| GET /api/v1/pollination/compatibility/parents | Names the knowledge base recognises, for type-ahead |
| POST /api/v1/pollination/compatibility/rank | Ranks candidate pollen donors for one pod parent |
| GET /api/v1/pollination/health | Model load state and class list |
| POST /api/v1/pollination/log/crosses | Records an attempted cross |
| POST /api/v1/pollination/log/crosses/{id}/stage | Advances a cross, or closes it at the gate at which it stopped |
| POST /api/v1/pollination/log/plants | Registers a plant, with or without a name |
| GET /api/v1/pollination/log/stats | Counts per gate, each beside its denominator |

**Table 2: Principal endpoints of Component 1. A further eight cover guidance, history, photograph storage and the stage vocabulary.**

Its artefacts are best_model.pkl, orchid_gate.pkl, flower_filter.pkl, preprocessors.pkl, trait_plant_strength.pkl and trait_leaf_condition.pkl, together with a knowledge base of 27 registered crosses, 11 hybrid genera and a table of parent traits. At runtime it writes the crossing logbook and its photographs under backend/data/pollination_log/, mirrored to the database under /pollination/.

## 5.2 Component 2 – Disease Detection and Severity Grading

This component classifies a leaf photograph into black leaf spot, phyllosticta leaf spot or healthy, grades the severity of a disease it has identified, recommends a treatment, and exposes the classifier in process to Component 1. Its router is disease_detection.py, supported by app/services/disease_service.py and by the model code under ml-models/disease_detection/src/.

The classifier is an EfficientNetB0, selected by a comparison that also considered a network trained from scratch and MobileNetV2. Its operating threshold of 0.70 was chosen from a sweep over the validation set rather than the test set. Severity is graded only for an image on which a disease was actually identified, so that a healthy leaf is never assigned a grade.

A contribution path allows a grower to submit a photograph of a disease the system does not recognise, and a further endpoint reports progress toward the threshold at which a new class may be trained. That threshold is a gate rather than a progress bar, and the distinction is deliberate: augmenting eight contributed images into several hundred files still carries eight images of information, and a class learned from so few examples draws a boundary that bleeds into its neighbours. The weakest existing class already sits at an F1 of 0.60, and an undertrained addition would lower it further, so the tooling refuses below the minimum rather than merely reporting the shortfall.

The component is built on TensorFlow 2.16.2 with Keras 3. Its six endpoints are given in Table 3.

| Method and path | Purpose |
|---|---|
| POST /api/v1/disease/detect | Uploads a leaf photograph and returns class, severity and treatment |
| GET /api/v1/disease/classes | The classes the system can recognise |
| GET /api/v1/disease/treatments/{name} | Treatment lookup, which requires no model |
| GET /api/v1/disease/status | Whether the disease and severity models are loaded |
| POST /api/v1/disease/contribute | Submits a photograph of an unrecognised disease |
| GET /api/v1/disease/pending-counts | Progress toward the retraining threshold |

**Table 3: Endpoints of Component 2.**

Its artefacts are disease_model.keras, severity_model.keras and a class manifest; contributed photographs accumulate under the component's pending directory with a manifest recording their disease, plant part and severity until they are verified. The route handlers are deliberately thin: every rule about when the severity model runs, and every threshold, resides in the model code so that the command line, the tests and the interface all apply the same logic.

### 5.3 Component 3 – Smart Watering and Automated Fertilisation

This component converts sensor telemetry into an irrigation and fertilisation plan, turns that plan into device commands, and owns the account, tenancy and farm schema on which the whole system rests. It is the largest surface in the deployment, spanning eight router modules and 74 endpoints across five prefixes, and supported by services for kriging, spatial analysis, tenant context and Firebase authentication.

Planning rests on kriging interpolation across a sparse sensor field, so that a section without a node of its own is planned from its neighbours rather than from defaults. A placement analysis path measures how far a single sensor's reading can be trusted across a house and recommends where the next node should be installed.

Two behaviours of this component are relied upon by the others. The first is the tenancy chokepoint: a single function rewrites every farm path onto the calling tenant and raises an error when no tenant is in context, rather than falling back to the shared tree. The second is a set of safety rules that refuse rather than guess, so that no command is issued while a node does not know the time, and no decision is taken from a reading more than two hours old.

The component is built on scikit-learn, PyKrige 1.7.2 for interpolation, TensorFlow for the visual hydration model, and firebase-admin 6.5.0. Its interface is summarised in Table 4.

| Method and path | Purpose |
|---|---|
| POST /api/v1/watering/predict | Runs both models and writes the result to the database |
| GET /api/v1/watering/prediction | The most recent prediction |
| GET /api/v1/watering/sensor-data | Current sensor snapshot |
| GET /api/v1/watering/history | Recent sensor history |
| POST /api/v1/watering/image-predict | Visual hydration class from a camera frame |
| /api/v2/care (45 endpoints) | Farm, houses, sections, plans, calibration, placement analysis and alerts |
| /api/v2/devices (10 endpoints) | Device registry, assignment, ping, scan and reporting interval |
| /api/v2/auto (9 endpoints) | Automation engine, automatic mode, overrides, alarms and push registration |
| /api/v2/accounts (5 endpoints) | Tenants, users and roles |

**Table 4: Endpoints of Component 3, with the larger prefixes summarised by count.**

Its two model artefacts, a watering regressor and a fertilisation model, are loaded from ml_pipeline/results/ beside the repository rather than from within it. They are consequently the only models that do not arrive with a repository pull, and Section 9.2 treats this as a deployment step. The component writes the farm subtree of the database, holding houses, sections, latest readings, plans, history, command queues and alarms.

## 5.4 Component 4 – Growth Stage Classification and Bloom Timing

This component recognises which of six growth stages a plant has reached and predicts how many days remain before it blooms. The six stages are vegetative, budding, pre-bloom, full bloom, wilting and seed formation, each carrying its own description and care protocol. Its routers are growth_stage.py and bloom_prediction.py, with model code under ml-models/growth_stage/src/ and ml-models/bloom_prediction/src/.

Beyond classifying a photograph of a single plant, the component offers an object-detection path which first crops a plant, flower bunch or seed pod out of a wider frame and then classifies each crop, so that a photograph of a whole bench may be used rather than only a photograph of one plant. A custom YOLOv8 detector covers those three classes, with a zero-shot detector retained for buds.

One finding from the development of this component is recorded here because it affects any retraining. Stage folders in the original dataset were found to be teaching the model contradictory labels, and a cleaning pass converted and quarantined the affected images. The correction is reported rather than silently applied, because a model retrained on the uncleaned data would not reproduce the figures in Section 14.

The component is built on TensorFlow 2.16.2 with Keras 3, Ultralytics 8.2.103 for the detector and Transformers 4.44.2 for the zero-shot path. Its eight endpoints are given in Table 5.

| Method and path | Purpose |
|---|---|
| POST /api/v1/growth/identify | Growth stage from a single-plant photograph |
| POST /api/v1/growth/identify-objects | Detects plants in a wider frame, then classifies each |
| GET /api/v1/growth/stages | The six stages and their descriptions |
| GET /api/v1/growth/stage/{key} | One stage with its care protocol |
| GET /api/v1/growth/health | Model load state |
| POST /api/v1/bloom/predict | Days to bloom from an image and environmental features |
| POST /api/v1/bloom/predict-objects | The same, per detected object in a wider frame |
| GET /api/v1/bloom/health | Model load state |

**Table 5: Endpoints of Component 4.**

Its artefacts are a growth model with its label encoder, a bloom model with its tabular scaler, and the detector weights. Training curves, confusion matrices and evaluation records are written to the outputs directory of each model.

### 5.5 Mobile Application and Field Hardware

The mobile application presents all four components on one device and collects the photographs and telemetry they depend upon. It is built on React Native 0.81.5 and Expo SDK 54, and organised into five tabs: My Farm for the watering component, Detect for disease, a central launcher, Hybrid for pollination and Growth for growth and bloom.

Two conventions in the application matter to the deployment. The backend address is written in exactly one file, having previously been copied into ten; relocating the backend then required editing all ten, and missing one produced a single screen failing silently while everything else continued to work. The interface key is sent only on endpoints that change something, and resides in a file excluded from version control, because the repository is public and the first key was published before that was noticed.

In the field, each microclimate zone carries an ESP32 node with a DHT22 sensor driving pump and tray relays, flashed from the firmware directory and wired according to the wiring guide held alongside it. A section is one zone and has exactly one device. Two browser-based simulators post the same payloads a node would, so that the watering component can be exercised where no hardware is present.

# 6 PREREQUISITES

The server requires Ubuntu 22.04 or later, although the backend is pure Python and also runs on Windows and macOS for development. Two virtual processor cores, four gigabytes of memory and twenty gigabytes of free disk are the practical minimum; the memory floor is set by TensorFlow and PyTorch being resident simultaneously, since the four components load fourteen model artefacts at process start. Python 3.10 or later is required, and the dependency set below was resolved against versions 3.11 and 3.12. The server needs outbound HTTPS to reach the Realtime Database, and a TLS certificate for its public hostname.

A development machine additionally requires Node.js 18 or later for the Expo development server, and an Android device carrying a build of Expo Go matching SDK 54, or an emulator. Development for iOS requires a Mac. The Arduino toolchain with the ESP32 board package is needed only when firmware is being flashed.

Several dependency versions are not free choices, and Table 6 records why. A saved model can only be unpickled by a compatible version of the library that produced it, so relaxing the numpy, scikit-learn or XGBoost pins yields a server that starts cleanly and then reports that no model is loaded on every prediction route. The full list is held in backend/requirements.txt.

| Package | Version | Reason for the pin |
|---|---|---|
| fastapi and uvicorn | 0.115.0, 0.30.0 | Application framework and server |
| numpy | 1.26.4 | The saved models were fitted against the 1.x interface |
| scikit-learn | 1.5.0 | Matches the version that serialised the gate and trait models |
| xgboost | 2.1.1 | Component 1's classifier is an XGBClassifier; version 3 requires numpy 2 |
| tensorflow | 2.16.2 | Keras 3 save format used by Components 2 and 4 |
| torch, torchvision | 2.3.1, 0.18.1 | Frozen ResNet18 embedding for Component 1 |
| ultralytics | 8.2.103 | YOLOv8 detector for Component 4 |
| transformers | 4.44.2 | Zero-shot detector, bud class only |
| PyKrige | 1.7.2 | Spatial interpolation for Component 3 |
| firebase-admin | 6.5.0 | Server-side database access |

**Table 6: Dependency versions that are constrained by the saved artefacts rather than chosen freely.**

# 7 DEPLOYMENT OPTIONS

Four configurations are in use, and Table 7 states the purpose of each. The hosted server is the deployed system and the address the mobile application points at by default. The remaining three exist for development: running the backend locally against a physical handset is how a change is tested in the field before it is deployed, the web build allows interface work with no device to hand, and the browser-based simulators exercise the watering component where no field hardware is present.

| Configuration | Purpose | Notes |
|---|---|---|
| Hosted virtual private server | The deployed system | Contabo server behind a public hostname. Deployment is a pull and a restart. |
| Local backend with a physical handset | Field-testing a change before deployment | Run the server bound to all interfaces and point the application at the machine's local address. |
| Local backend with the web build | Interface work with no device available | The web build reaches the backend on the loopback address. |
| Simulated field hardware | Exercising Component 3 without an ESP32 | The two browser simulators post the payloads a node would. |

**Table 7: Deployment configurations.**

One caution applies to the second configuration. A local address is a lease rather than a fixed property of the machine: during development this server was assigned one address and then a different one on the next reconnection, and in the application that change presents only as a failed network request. The current address should be confirmed before the application is suspected.

# 8 QUICK START

From a clean checkout on the server, the backend is brought up as follows.

```
git clone https://github.com/dinith114/R26-SE-018.git
cd R26-SE-018/backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

The service prints the load state of every model as it starts. A component whose artefacts are absent logs a warning and its routes answer with a service-unavailable status, while the remaining three continue to serve; this is the intended behaviour rather than a partial failure. Interactive documentation of the whole interface is published at /docs.

The mobile application is started from a development machine as follows.

```
cd R26-SE-018/mobile
npm install
npx expo start
```

# 9 STEP-BY-STEP INSTRUCTIONS

## 9.1 Deploying the backend

Pull the branch being deployed and confirm that the working tree is clean. Activate the virtual environment and install requirements, although this step is needed only when the dependency file has changed, since reinstalling TensorFlow and PyTorch takes several minutes. Confirm that the model artefacts are present, noting the distinction drawn in Section 9.2. Restart the service, which under systemd is a single command and under a development runner happens automatically. Finally, run the checks of Section 13.1 before reporting the deployment complete.

## 9.2 Artefacts that do not arrive with a pull

Twelve model files are tracked in the repository and are present after a clone. Two are not, and a deployment that overlooks them produces a server which starts cleanly and then refuses one component's predictions. Table 8 states which is which.

| Component | Artefacts | Source |
|---|---|---|
| Component 1 | best_model.pkl, orchid_gate.pkl, flower_filter.pkl, preprocessors.pkl, trait_plant_strength.pkl, trait_leaf_condition.pkl | Tracked; arrives with the pull |
| Component 2 | disease_model.keras, severity_model.keras | Tracked; arrives with the pull |
| Component 4 | vanda_growth_model.h5, label_encoder.pkl, vanda_bloom_model.h5, tabular_scaler.pkl | Tracked; arrives with the pull |
| Component 3 | best_model.pkl, best_fertilization_model.pkl | Not tracked. Expected at ml_pipeline/results/ beside the checkout, and copied to the server |

| | | |
|---|---|---|
| | | separately |

**Table 8: Model artefacts that ship with the repository, and those that do not.**

The ignore rules exclude serialised models as a class and then readmit the twelve files above by name. A model newly added to a component is therefore invisible to version control until it is named as an exception, and Component 3's two reside outside the tree altogether.

### 9.3 Writable paths

One directory is created at runtime rather than tracked. The crossing logbook and the parent photographs uploaded to it are written under backend/data/pollination_log/, which the service creates on first use, so the only requirement is that the process may write beneath backend/data/. It is excluded from version control because it holds a grower's own records and photographs, and the repository is public.

### 9.4 Deploying the mobile application

Set the backend address in the single configuration file that holds it, which for a release is the public hostname and for field testing the local address of the development machine. Create the secret file carrying the interface key, which a fresh checkout will not contain because it is excluded from version control. Start the development server and scan the presented code with Expo Go, or build natively for a release. The installed Expo Go must match SDK 54, since a newer build will refuse the bundle.

### 9.5 Commissioning field hardware

Flash the section-node firmware to the ESP32 and wire the sensor according to the wiring guide. Register the device through the device endpoints and assign it to a section, remembering that a section is one microclimate zone and has exactly one device. Confirm that readings are arriving before enabling automatic watering, because the automation engine refuses to act on a reading more than two hours old and will otherwise appear to do nothing.

# 10 SERVICE AND PORT MAP

Only one port need be reachable from outside the development network. Table 9 lists the services and their exposure.

| Service | Port | Exposure | Notes |
|---|---|---|---|
| FastAPI backend | 8000 | Public, behind TLS | All 109 endpoints. Bind to all interfaces to reach it from a handset. |
| Expo development server | 8081 | Local network | Bundler; development only |
| Expo developer tools | 19000–19002 | Local network | Only while the development server is running |
| Realtime Database | 443 | Outbound | Reached over the REST interface |
| ESP32 nodes | None | Local network | Nodes poll outward; no inbound port is opened on them |

**Table 9: Services and ports.**

On a Windows development machine the inbound rule for the backend port must be added once, in an elevated shell.

```
netsh advfirewall firewall add rule name="Uvicorn 8000" dir=in
action=allow protocol=TCP localport=8000
```

# 11 INTEGRATION GUIDE

Four hand-offs cross a component boundary; everything else is internal to one component and is documented in that member's individual thesis. Table 10 states each, together with its behaviour when the other side is unavailable, because that behaviour is as much a part of the contract as the success path.

| From | To | Mechanism | Behaviour on failure |
|---|---|---|---|
| C1 | C2 | In-process call through the disease provider | The provider reports zero confidence, which is distinct from, and never collapsed into, a report of no disease |
| C1 | Database | Crossing logbook subtree | Local storage is authoritative; the mirror is best-effort and its failure cannot fail a save |
| C3 | All | Tenancy chokepoint and the database helpers | The chokepoint raises when a farm path is used with no tenant in context, rather than falling back to the shared tree |
| C3 | Devices | Command queue beneath the section subtree | No command is issued while a node does not know the time, and no decision is taken from a reading over two hours old |

**Table 10: Cross-component hand-offs.**

The tenancy property is asserted rather than assumed. A test scans every backend source file for a line that names the database without passing through the agreed helpers and fails the build where it finds one. It caught a genuine violation in the crossing logbook during development, which no behavioural test would have detected because the call in question succeeded.

# 12 CONFIGURATION

Table 11 lists the settings that are changed rather than merely read.

| Setting | Location | Purpose |
|---|---|---|
| Backend address | mobile/src/config/backend.js | The single place the application's base address is written |
| Interface key | mobile/src/config/secret.js | Sent on endpoints that change something; excluded from version control |
| Server key | Server environment | The server side of the same shared secret |
| Service-account path | Server environment | Credentials for server-side database access |
| Database root | Backend watering router | Realtime Database address |
| Gate threshold, 0.25 | Component 1 model directory | Refusal distance, chosen from the keep-fraction quantile |
| Minimum sample, 5 | Component 1 logbook service | Completed attempts required at a gate before a rate is shown |

**Table 11: Configuration settings.**

One practice is recorded because it was learned at cost. The repository is public, and the first interface key was committed and published before that was noticed. It was rotated, the file holding it was excluded from version control, and no key belongs in a tracked file. This should be confirmed before any release.

# 13 VERIFICATION AND TESTING

## 13.1 Checks after a deployment

Four requests, in order. Each fails differently, which is the reason for running all four rather than only the first.

```
curl https://orchidfarm.duckdns.org/api/v1/pollination/health
curl https://orchidfarm.duckdns.org/api/v1/pollination/log/stats
```

```
curl https://orchidfarm.duckdns.org/api/v2/care/model-info
curl https://orchidfarm.duckdns.org/docs
```

The first confirms that Component 1's classifier is loaded and reports its classes. The second confirms that the logbook store is readable and writable. The third returns the metrics and baselines of the deployed models, so that the figures shown in the application are read from the models rather than transcribed. The fourth returns the interactive schema, which demonstrates that every router mounted successfully.

### 13.2 Automated suite

The backend carries 327 test functions across 29 suites, run by continuous integration on every push and pull request, and a failing test blocks the merge. Table 12 states what the principal categories establish.

| Category | What it establishes |
|---|---|
| Tenancy chokepoint | No backend file reaches the database outside the agreed helpers |
| Tenant propagation | A request touches the caller's tenant and no other |
| Gate behaviour | A non-orchid image produces a structured refusal carrying the gate's own measurements |
| Trait precedence | A grower's correction overrides a measured value, and an unanswered trait is omitted rather than sent as a placeholder |
| Provider availability | The disease provider reports itself unavailable when the runtime cannot be imported, not merely when the file is absent |

**Table 12: Principal verification categories.**

Two suites require optional dependencies that are not installed on every development machine: the placement tests require the kriging library, and the command-status tests require authentication configuration. Both pass in continuous integration. A local run lacking them reports failures unrelated to the change under test, and this is noted because it has caused confusion.

# 14 MACHINE-LEARNING MODEL REFERENCE

Table 13 states the performance of each deployed model beside the reference value it must exceed, and the split on which it was measured. Every figure is reported with its baseline, including the unfavourable ones.

| Component | Task | Result | Reference | Split |
|---|---|---|---|---|
| C1 | Pollination suitability, three classes | Accuracy 0.717, macro F1 0.715 | Majority baseline 0.639 | Grouped by plant, n = 357 |
| C1 | Input gate, non-orchid refusal | 91.9 per cent refused, AUC 0.921 | 309 of 309 non-plant images refused | Held-out validation, n = 455 |
| C2 | Disease classification, three classes | Accuracy 0.806, macro F1 0.778 | Majority baseline 0.418 | Held-out test, n = 67 leaves |
| C2 | Severity grading, three grades | Macro F1 0.450 | Majority baseline 0.395 | Held-out test, n = 43 leaves |
| C3 | Watering time | MAE 8.42 min, $R^2$ 0.954 | Label noise floor 8.62 min | Out-of-time, n = 14,620 |
| C3 | Humidity below 60 per cent within three hours | AUC 0.986 | No-skill AUC 0.500 | Out-of-time, n = 47,515 hours |
| C4 | Growth stage, six classes | Accuracy 0.940, macro F1 0.94 | Majority baseline 0.196 | Held-out test, n = 1,768 |
| C4 | Bloom timing | MAE 0.92 days | Mean-prediction MAE 4.86 days | Held-out test, n = 1,839 |

**Table 13: Deployed model performance, each figure beside its reference value.**

## 14.1 Why no success probability is produced for a cross

The registered orchid record lists hybrids that were made and registered. No breeder registers an attempt that produced no pod, so the record carries no denominator and no honest success rate can be derived from it. The source material contradicts itself on precisely this point: a compilation derived from the register describes intergeneric crosses as having moderate to high success, while the breeder consulted for this project reports that most intergeneric attempts do not take. Both statements are honest, because the register sees only the survivors. The deployed system therefore returns an evidence tier with the precedents behind it, and the crossing logbook exists to collect the failures the register structurally cannot hold.

# 15 TROUBLESHOOTING

Table 14 records the symptoms observed during development, their causes, and their resolutions. Each was encountered in practice rather than anticipated.

| Symptom | Cause | Resolution |
|---|---|---|
| A prediction route reports that no model is loaded | A dependency version was relaxed, so the serialised model cannot be read by the installed library | Reinstall from the requirements file without upgrading, checking numpy, scikit-learn and XGBoost first |
| The application reports a failed network request | The configured address is no longer that of the machine, or the inbound firewall rule is absent | Confirm the current address, confirm the server was bound to all interfaces, and add the rule |
| The disease reading is always absent | The deep-learning runtime cannot be imported on the server | The provider is reporting correctly. Install the runtime rather than suppressing the warning |
| Expo Go refuses the bundle | The installed Expo Go is a later SDK | Install the build matching SDK 54, or produce a development build |
| A section is never watered | No reading has arrived, or every reading is older than two hours | Confirm the node is reporting. The engine refuses stale telemetry by design |
| Logbook entries do not reach the database | Outbound HTTPS is blocked, or the database rules reject the write | The local record is already safe. Check connectivity, then the rules on that subtree |
| The tenancy test fails in continuous integration | New code constructs a database address itself | Route the call through the agreed helpers |

**Table 14: Symptoms encountered during development and their causes.**

# 16 CLOUD DEPLOYMENT

The system is deployed on a virtual private server rather than a managed container platform. The reason is proportionality: the backend is one Python process with no horizontal scaling requirement at pilot scale, and a managed platform would add an orchestration layer to a system that does not yet need one.

The public hostname is provided by a dynamic naming service, which gives a stable name to an address that would otherwise change. Transport security terminates in front of the application, so the server itself binds plain HTTP on its port and is not exposed directly. The Realtime Database is the only managed service in the stack and is reached over its REST interface, so the server requires outbound HTTPS and nothing further. Most model artefacts are tracked in the repository rather than fetched from object storage, so a deployment needs no credentials beyond those already present in the environment; Component 3's two are the exception, as Section 9.2 sets out.

One matter remains outstanding before a production rollout. Firebase Authentication is not yet enabled on the project, and until it is there is no authenticated user and therefore no identifier onto which tenant and role claims can be stamped. The first-tenant provisioning script in the repository is deliberately unrun for this reason. Enabling authentication is the first step of any rollout beyond the pilot.

# 17 MAINTENANCE AND OPERATIONS

A routine deployment consists of a pull, a restart and the four checks of Section 13.1. No migration step exists, because the database is schemaless and the logbook store creates itself. Updating a model means replacing the artefact, restarting, and confirming through the health endpoint that it loaded; retraining procedures are documented in each component's own directory.

Backups must cover two stores rather than one. The Realtime Database holds the authoritative copy of farm and tenant data, while the local logbook file is

authoritative for crossing records and is mirrored to the database rather than the reverse. Both therefore belong in any backup policy.

Logs should be read with one expectation in mind: a component reporting itself unavailable is the system working as designed, not failing. The service prints the load state of every model at start and warns on each failed database call. Finally, the dependency versions of Table 6 should be treated as frozen until the models are refitted; everything else may move freely.

# 18 SCALING

At pilot scale the system serves a small number of growers across a handful of farms, for which a single process is more than sufficient. Table 15 states the constraints that would be reached first, in the order in which they would arrive.

| Constraint | What reaches it first | Response |
|---|---|---|
| Memory | TensorFlow, PyTorch and the detector weights resident together | Separate the image components into a second process, or move inference behind a queue |
| Processor time during inference | Concurrent image assessments, each a forward pass through ResNet18 followed by a tree ensemble | Add workers. The application is stateless, so this requires only a load balancer |
| Database growth | Sensor history accumulating without bound | Age out raw readings and retain aggregates; the planners already read windows rather than whole histories |
| Photograph storage | Two photographs per logged cross, retained indefinitely | Move photographs to object storage and retain the records, which are a few hundred bytes each |

**Table 15: Scaling constraints in the order they would be reached.**

# 19 SUPPORT AND RESOURCES

Table 16 lists the material a reader is most likely to need next.

| Resource | Location |
|---|---|
| Source repository | github.com/dinith114/R26-SE-018 |
| Deployed interface | https://orchidfarm.duckdns.org |
| Interactive schema | https://orchidfarm.duckdns.org/docs |
| Backend setup | backend/README.md |
| Component 1 method and measurements | ml-models/hybrid_pollination/README.md |

| | |
|---|---|
| Field node setup | iot/README.md and firmware/WIRING_GUIDE.md |
| Component methods in full | The individual thesis of each component owner |

**Table 16: Supporting material.**

# 20 CONCLUSION

The deployed system is one FastAPI process serving 109 endpoints to a React Native application, supported by a Realtime Database and a directory of model artefacts, with ESP32 nodes reporting from the shade house. Four research components share that process while retaining their boundaries: each owns a router prefix, its own models and its own subtree, and the four points at which one reaches into another are enumerated and tested.

The deployment decisions that matter most are not the conspicuous ones. Pinning three numerical libraries is what keeps the saved models loadable. Routing every database path through a single function is what keeps one farm's data from reaching another, and the test asserting it caught a real violation during development. Writing logbook entries locally before mirroring them is what makes the system usable in a shade house without signal. And a component that reports itself unavailable rather than guessing is the behaviour on which the whole integration depends.

What remains before a production rollout is stated plainly. Authentication is not yet enabled, and until it is there is no user onto which tenant claims can be stamped. Everything else described in this report is running.